Project Abstract Submission

An Interactive Context-Free Grammar Analyzer for Parse Tree Generation, Ambiguity Detection, and Grammar Transformation

By

Bindu Potukuchi 2420090092
Sarvani Topella 2420030367
Tanvi Bandaru 2420030507
Varshitha Ravuri 2420030536

Under the guidance of

Dr. Krishna Kishore

Course: Compiler Design | A.Y. 2026 – 2027

Department of Computer Science and Engineering

Introduction

- Context-free grammars (CFGs) form the theoretical backbone of compiler design and language processing.
- Learners frequently struggle to see how a grammar generates parse trees — and, more importantly, why a grammar may be ambiguous.
- Existing classroom tools typically produce a single static parse tree for a given input, with no insight into ambiguity, its cause, or how the grammar could be rewritten to remove it.
- This gap forces students to reason about grammar transformations — left factoring, left-recursion elimination, precedence assignment — purely on paper, with no interactive feedback.

Abstract

- **Interactive analyzer:** a user inputs a CFG and a test string; the system validates the grammar's structure and generates the corresponding parse tree.
- When an input string admits more than one valid derivation, the analyzer generates and displays all possible parse trees and flags the grammar as ambiguous.
- Each derivation is classified — leftmost, rightmost, parse tree, or abstract syntax tree, where applicable.
- **Explains the ambiguity** by highlighting where the derivations diverge, and suggests concrete fixes: left factoring, left-recursion elimination, or precedence/associativity rules.
- A before-and-after comparison re-parses the input against the transformed grammar to visually confirm the ambiguity has been removed.

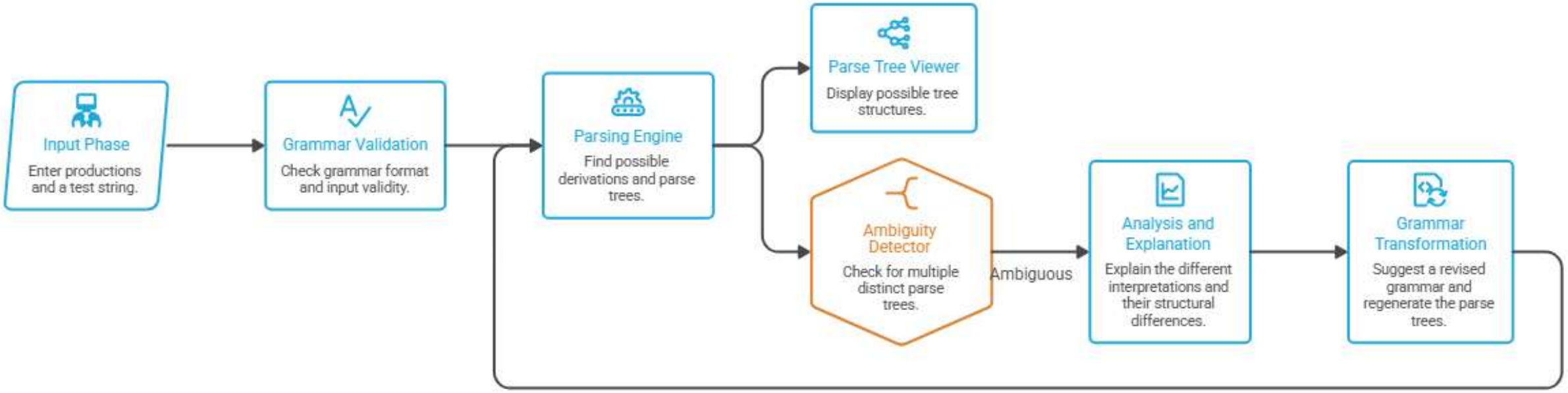
Limitations of Existing Tools

- Produce only a single, static parse tree per input — no handling of ambiguous grammars.
- Offer no explanation of why a grammar is ambiguous or where the derivations diverge.
- Provide no guidance on how an ambiguous grammar could be rewritten.
- Leave grammar transformations — left factoring, left-recursion elimination, precedence assignment — as pen-and-paper exercises, with no interactive feedback.

Proposed System — Objectives

- Accept a user-defined context-free grammar and a test string, and validate the grammar's structure.
- Generate the corresponding parse tree(s) for the input string.
- Detect ambiguity by finding and displaying all valid derivations, classified as leftmost, rightmost, parse tree, or AST.
- Explain the source of ambiguity by highlighting exactly where the derivations diverge.
- Suggest concrete transformations — left factoring, left-recursion elimination, or precedence/associativity rules.
- Re-parse the input against the revised grammar to visually confirm the ambiguity is resolved.
- Remain extensible toward LL/LR parse table construction and code generation.

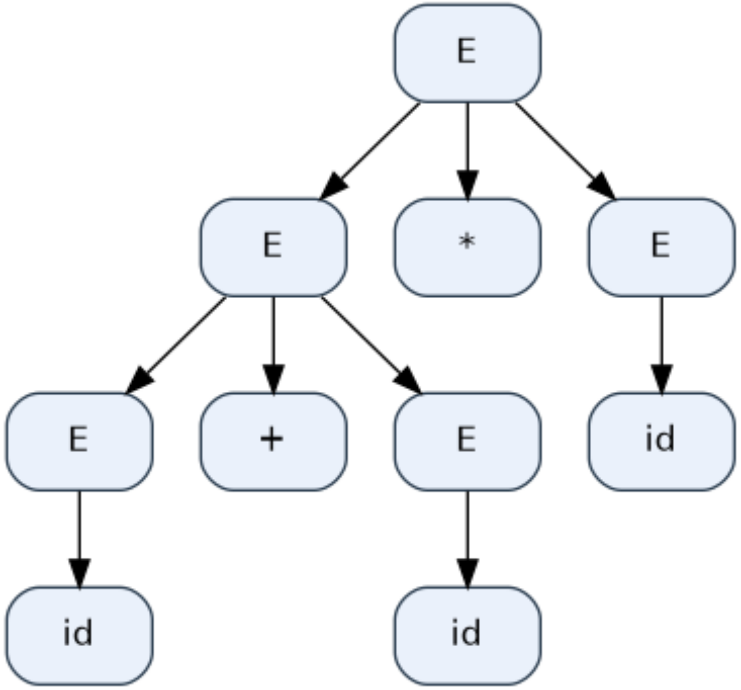
System Architecture



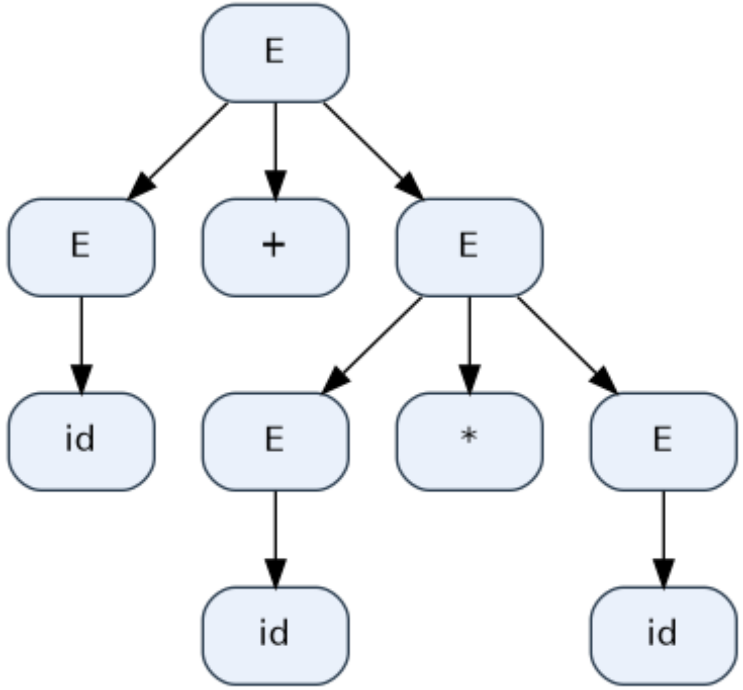
Illustrative Example — Ambiguous Grammar

Grammar: E → E + E | E * E | id

Input string: id + id * id



*Tree 1: (id + id) * id*



*Tree 2: id + (id * id)*

Ambiguity Analysis & Suggested Fix

⚠ Ambiguous Grammar

The input string has multiple valid parse trees.

Reason: the grammar does not specify precedence between '+' and '*'.

Suggested solution: define '*' with higher precedence than '+'.

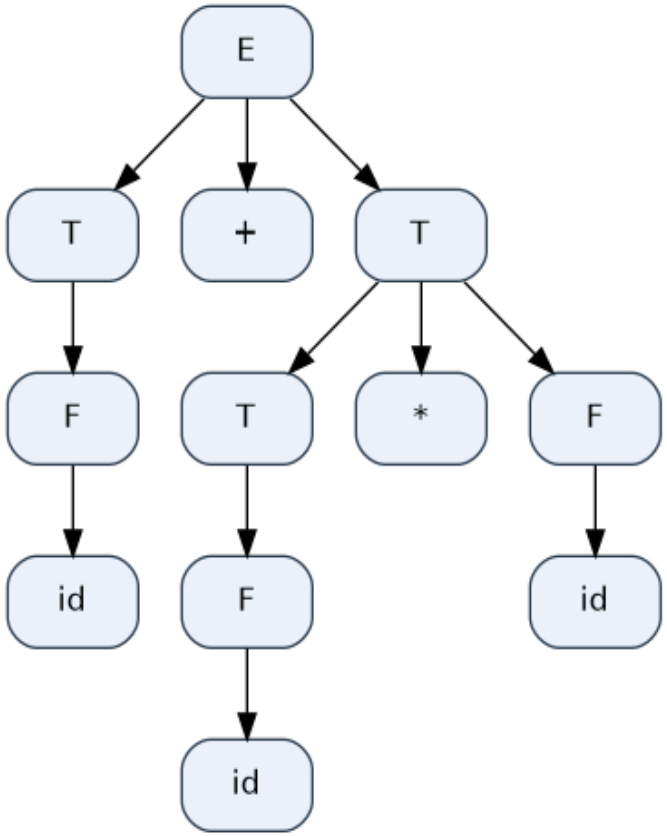
Revised Grammar

E → E + T | T

T → T * F | F

F → id

Resolved Parse Tree



Conclusion & Future Scope

- The Grammar Analyzer turns an abstract compiler-design exercise into an interactive, visual experience.
- It validates CFGs, generates parse trees, detects ambiguity, and explains its underlying cause.
- It suggests concrete grammar transformations and verifies them by re-parsing the revised grammar.
- **Future scope:** extend the analyzer to LL/LR parse table construction and code generation.